

14. Ejercicio de CoT
Explicación de BookingCalendar

Jonathan Ricardo Serna Galina

20074902

1. ¿Qué es CoT y cómo se usa en un prompt?

Introducción

En este trabajo se explica cómo funciona el programa BookingCalendar y cómo se puede organizar una solicitud a la IA para resolver la actividad. El proyecto se desarrolló con asistencia de IA. La explicación siguiente describe el código y ejemplos comprobables; no es una transcripción del razonamiento interno del modelo.

La idea de Chain of Thought

CoT significa Chain of Thought, o cadena de pensamiento. Wei y sus colaboradores estudiaron una técnica que incluye ejemplos de preguntas con pasos intermedios y una respuesta final. El modelo recibe ese patrón dentro del prompt. En sus experimentos, este enfoque mejoró el rendimiento en varias tareas de razonamiento aritmético, simbólico y de sentido común [1].

Dos formas que aparecen en la investigación

Few-shot CoT: se proporcionan algunos ejemplos resueltos con pasos intermedios. **Zero-shot CoT:** no se incluyen ejemplos resueltos. Kojima y sus colaboradores estudiaron una instrucción breve, "Let's think step by step", para inducir este tipo de respuesta y encontraron mejoras en los modelos y tareas evaluados [2].

Aplicación a esta tarea

Para este ejercicio, una forma práctica de organizar el prompt es pedir requisitos, diseño de la clase, reglas de horarios, código y pruebas. Esta organización sirve para revisar la respuesta. Por sí sola no demuestra que se haya reproducido el método experimental de los artículos.

Qué se debe comprobar

Una explicación ordenada no sustituye la ejecución. En BookingCalendar, lo importante es comprobar que una reserva ocupada se rechace, que una cancelación correcta funcione y que la tabla muestre los espacios disponibles. Por eso se acompañó el programa con pruebas y salidas reales.

2. Prompt propuesto para la actividad

Contexto del prompt

El siguiente prompt es una versión mejorada para volver a solicitar la solución. Se incluye como propuesta de la revisión del trabajo, no como copia de un mensaje enviado durante la primera generación.

Solicitud

“Ayúdame a resolver una actividad universitaria de Python. Necesito una clase BookingCalendar para reservar salas de reuniones en un mismo día. Revisa el archivo cot_test.py y conserva los nombres de clase y métodos que utiliza.”

Reglas del programa

“Cada reserva debe tener inicio, fin y sala. book debe devolver True si logra reservar y False si los datos son inválidos o hay traslape en esa sala. cancel debe devolver True solo cuando se elimina la reserva exacta. get_available_slots debe mostrar los huecos libres en una tabla. Usa formato de 24 horas y sigue PEP 8.”

Formato de respuesta

“Presenta una explicación de la solución en este orden: 1) requisitos; 2) datos de la clase; 3) validación de horarios; 4) detección de traslapes; 5) cancelación; 6) cálculo de disponibilidad; 7) diagrama de clases y código; 8) resultados de pruebas. Explica cada parte con un ejemplo pequeño y distingue resultados esperados de resultados ejecutados.”

Ejemplos que aclaran la solicitud

“En Sala A, 09:00-11:00 y 11:00-12:00 deben aceptarse porque sus horarios solo se tocan en el límite. En la misma sala, 10:00-12:00 debe rechazarse si ya existe 09:00-11:00. En Sala B, 09:00-11:00 debe poder reservarse aunque Sala A esté ocupada.”

Por qué el prompt es más claro

Especifica qué se necesita, qué nombres debe tener el código y qué resultados se esperan. Los ejemplos aclaran la diferencia entre horarios consecutivos y horarios que se enciman. También pide pruebas para poder revisar si el programa cumple con la actividad.

3. Explicación de la solución: datos y reservas

Paso 1. Definir las operaciones

La clase tiene tres operaciones principales: reservar, cancelar y consultar disponibilidad. `book` y `cancel` regresan valores booleanos. Esto permite comprobar el resultado directamente, como hace el archivo del profesor con `assert`.

Paso 2. Separar las salas

El atributo `_bookings` es un diccionario. Cada nombre de sala tiene su propia lista de reservas. Por ejemplo, Sala A puede tener `[(540, 660)]` y Sala B otra lista independiente. Así, una reunión en Sala A no bloquea Sala B.

Paso 3. Representar las horas

El programa convierte las horas a minutos desde medianoche. 09:00 equivale a $9 \times 60 = 540$ minutos; 11:00 equivale a 660. También admite minutos: 09:15 equivale a $9 \times 60 + 15 = 555$. Al imprimir, convierte esos valores nuevamente a HH:MM.

Paso 4. Validar antes de guardar

El nombre de la sala no puede estar vacío y el inicio debe ser menor que el fin. Por eso 14:00-14:00 y 14:00-12:00 devuelven `False`. Se aceptan enteros para que funcionen las llamadas del profesor y cadenas HH:MM para horarios con minutos. 24:00 se permite solo como fin.

Paso 5. Detectar un traslape

La condición usada es: `inicio_nuevo < fin_existente` y `inicio_existente < fin_nuevo`. Si ambas comparaciones son verdaderas, los intervalos comparten tiempo. Por ejemplo, 10:00-12:00 y 09:00-11:00 cumplen $600 < 660$ y $540 < 720$: la nueva reserva se rechaza.

Paso 6. Permitir horarios consecutivos

Si la nueva reserva es 11:00-12:00 y la anterior es 09:00-11:00, la primera comparación es $660 < 660$, que es falsa. No hay traslape. Si la reserva es válida, se agrega a la lista de su sala, se ordena y `book` devuelve `True`.

4. Explicación de la solución: cancelación y tabla

Paso 7. Cancelar una reserva exacta

cancel busca el mismo inicio, fin y nombre de sala. Si existe 09:00-11:00 en Sala A, cancelar 09:00-10:00 no la elimina. Tampoco funciona cancelar el mismo horario en otra sala. Solo la coincidencia exacta se elimina y devuelve True.

Paso 8. Obtener los espacios libres

La consulta recorre las reservas ordenadas y muestra los espacios que quedan entre ellas. La jornada del proyecto va de 00:00 a 24:00. Para las reservas 08:00-10:00, 10:00-12:00 y 14:00-16:00, los huecos son 00:00-08:00, 12:00-14:00 y 16:00-24:00.

Paso 9. Mostrar la tabla

get_available_slots imprime las columnas Sala, Inicio y Fin. Además, devuelve una lista con esas mismas filas. Esto permite ver el resultado en consola y también compararlo con el resultado esperado en una prueba.

Paso 10. Revisar los casos especiales

Una sala ocupada de 00:00 a 24:00 no tiene huecos. Si se cancela su última reserva, queda libre todo el día. Una sala nueva consultada por nombre se muestra libre sin guardarla en el calendario. Los nombres distinguen mayúsculas, pero se eliminan espacios al principio y al final.

Ejemplo de disponibilidad

Sala	Inicio	Fin
Sala A	00:00	08:00
Sala A	12:00	14:00
Sala A	16:00	24:00

Tabla ilustrativa del ejemplo del paso 8. Si se cancela 14:00-16:00, los dos últimos huecos se unen y queda libre 12:00-24:00.

5. Comprobación y conclusión

Paso 11. Probar el programa

En la ejecución documentada el 15 de septiembre de 2026, el archivo `cot_test.py` terminó correctamente y las 13 pruebas adicionales de unittest finalizaron con OK. La revisión de estilo con `pycodestyle` no reportó errores en los tres archivos nuevos. Esta revisión del documento no modifica el código.

Detalle encontrado en las pruebas del profesor

Dos líneas del archivo `cot_test.py` no tienen `assert`. Son las llamadas que usan 14-14 y 14-12. Por ello no comprueban realmente que esos horarios se rechacen. El archivo original se conservó y las pruebas adicionales verifican que ambos casos devuelvan `False`.

Qué aportan las pruebas adicionales

Se revisan reservas consecutivas, distintas formas de traslape, salas independientes, datos inválidos, horarios con minutos, cancelación exacta, orden de los huecos y formato de la tabla. También se comprueban una sala ocupada todo el día y una sala todavía no registrada.

Conclusión

El punto principal del ejercicio es comparar correctamente los intervalos y mantener separadas las salas. Explicar cada parte ayuda a entender el código, pero las pruebas son las que permiten verificar su comportamiento. El programa cumple las operaciones solicitadas y puede ejecutarse sin instalar librerías externas. Sus datos se borran al cerrar la ejecución porque se guardan en memoria.

Referencias

[1] Wei, J., et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. Primera versión publicada en 2022.

<https://arxiv.org/abs/2201.11903>

[2] Kojima, T., et al. (2022). *Large Language Models are Zero-Shot Reasoners*.

<https://arxiv.org/abs/2205.11916>

Fuentes consultadas el 19 de septiembre de 2026.

Liga del código

[Abrir BookingCalendar en GitHub](#)